

Справочник объектов

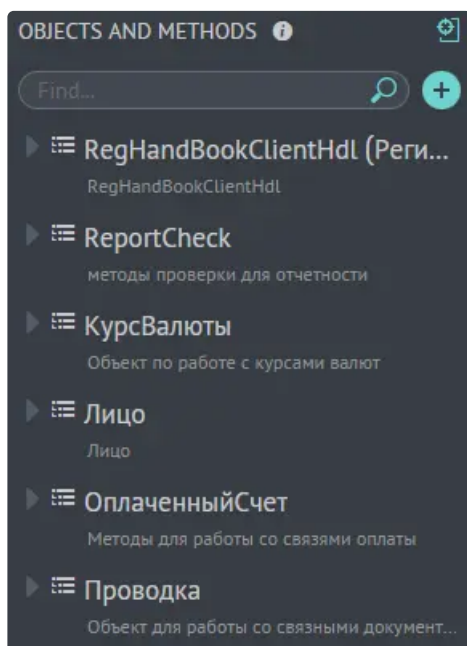
Справочник объектов — это одна из абстракций Wasaby Framework, которая содержит описание [объектов](#) и [методов](#), при помощи которых можно организовать бизнес-логику веб-приложения. В одном справочнике может быть описано несколько объектов. Имена объектов должны быть уникальны внутри одного справочника.

Физически файл справочника объектов представляет собою XML-документ с расширением *.ogx; файл простой иерархической структуры, в которой можно выделить две категории объектов:

- описания объектов бизнес-логики (они же организуют иерархию);
- описания методов объектов БЛ (листовые элементы иерархии).

Подробнее о формате файла можно прочитать [здесь](#).

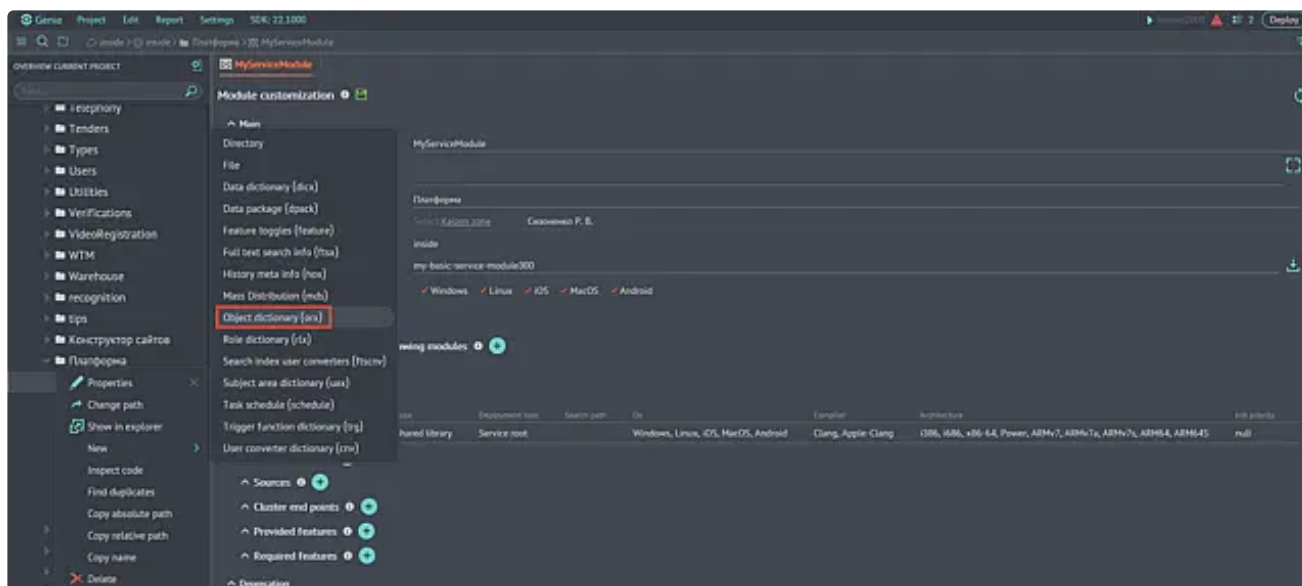
Справочник объектов позволяет наглядно показать объектную модель предметной области.



Возможность вести комментарии к объектам и методам объектов позволяет задокументировать интерфейс сервиса бизнес-логики.

Создание справочника

Создание справочников производят через контекстное меню сервисных модулей (см. [Левая панель. Вкладка открытого проекта](#)):



Общее описание формата файлов справочников объектов

В целом, файл справочника объектов представляет собою XML-документ простой иерархической структуры, в которой можно выделить две категории объектов:

- описания объектов бизнес-логики (они же организуют иерархию);
- описания методов объектов БЛ (листовые элементы иерархии).

Формат файлов справочников объектов *.orx

```

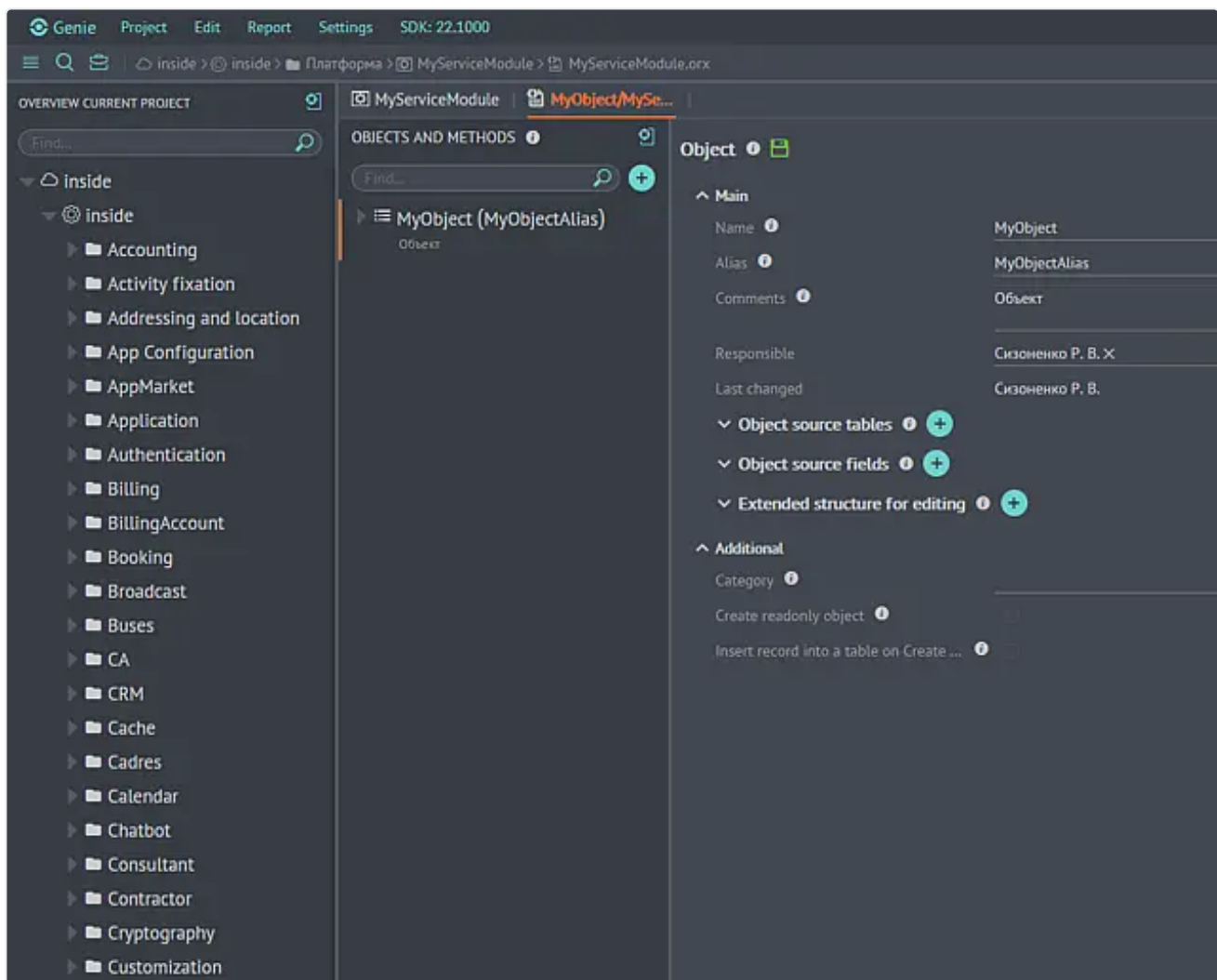
1 <repository orx_version="1">
2   <!--Описания объекта репозитория -->
3   <!--Описания методов базового объекта -->
4 </repository>
```

Корень репозитория считается базовым объектом, от которого порождены все другие объекты. Атрибут **orx_version** задает версию формата файла репозитория.

Формат описания активного объекта

```

1 <object name="MyObject" alias_name="MyObjectAlias"
  last_changed="Иванова А.И." last_changed_uuid="3f71a247-c130-4ca5-8497-
  87d107dc08f0" responsible="Иванова А.И." responsible_uuid="3f71a247-
  c130-4ca5-8497-87d107dc08f0">
2   <!-- Структура записи объекта -->
3   <comment>Описание объекта</comment>
4   <!-- Описания вложенных объектов -->
5   <!-- Описания собственных методов объекта -->
6 </object>
```



Как видим, объект является контейнером для других объектов и методов. Его собственными атрибутами являются:

1. **name** – имя объекта (например, «Лицо»);
2. **alias_name** – псевдоним объекта; альтернативное название, по которому объект доступен;
3. **read_only** – 0 или 1, является ли объект объектом только для чтения (т.е. для него автоматически генерируется только метод «Прочитать») или же для него генерируется полный набор методов. Имеет смысл лишь в том случае, если указана структура записи объекта;
4. **insert_on_create** – флажок «Создавать в БД» — задает поведение метода «Создать», имеет смысл только для объектов со структурой записи и не только для чтения;
5. **comment** – текстовое описание объекта, комментарии пользователя;
6. **responsible** – ФИО ответственного разработчика;
7. **responsible_uuid** – uuid ответственного разработчика;
8. **last_changed** – ФИО разработчика, внесшего изменения в объект;
9. **last_changed_uuid** – uuid разработчика, внесшего изменения.

На данный момент существуют следующие категории методов:

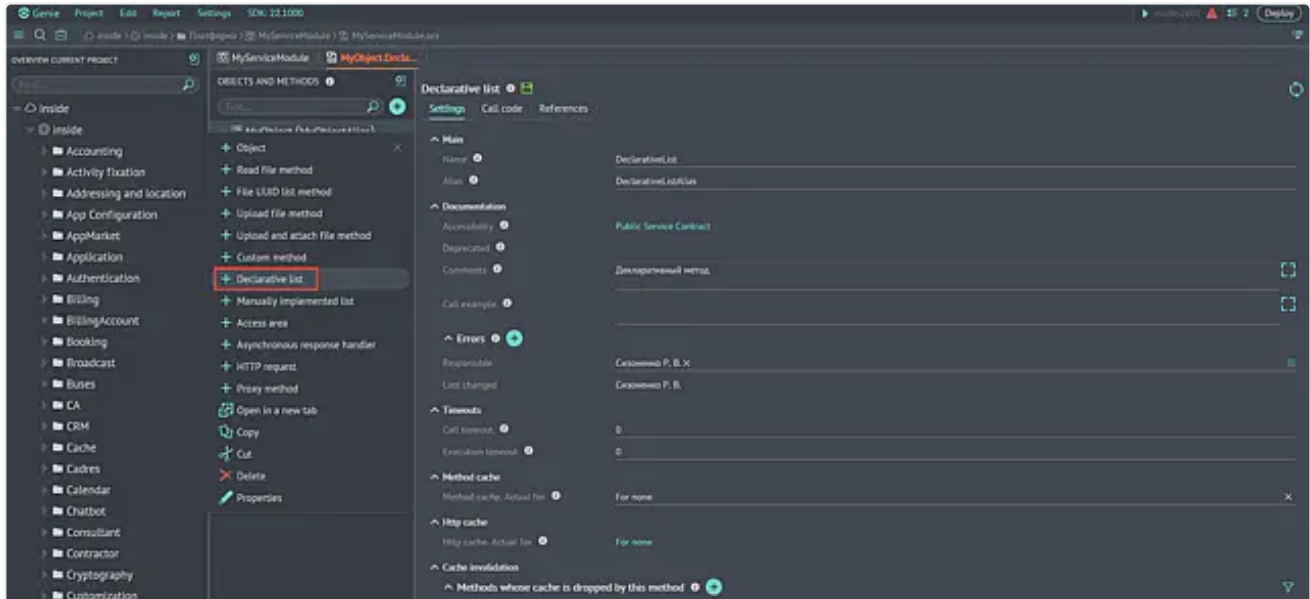
1. [Автогенерируемые методы](#).
2. [Методы, описанные декларативно](#) (упрощенные стандартные методы) – принимают на вход четыре параметра и возвращают список записей.
3. [Методы, реализуемые вручную](#) (стандартные методы) – обычные ХП/методы C++, сигнатура которых совпадает с сигнатурой методов, описанных декларативно.
4. [Произвольные методы](#) – ХП/функции C++ с произвольной сигнатурой и возвращаемым значением.
5. [Методы для работы с файлами](#):
 - [Методы для чтения файла](#);
 - [Методы для чтения списка файлов по UUID](#);
 - [Методы для загрузки файла на сервер](#);

- [Методы для загрузки файла на сервер с одновременным прикреплением к объекту БЛ.](#)

6. Удаленные вызовы:

- [Прокси-метод;](#)
- [HTTP-запрос.](#)

7. Область видимости.



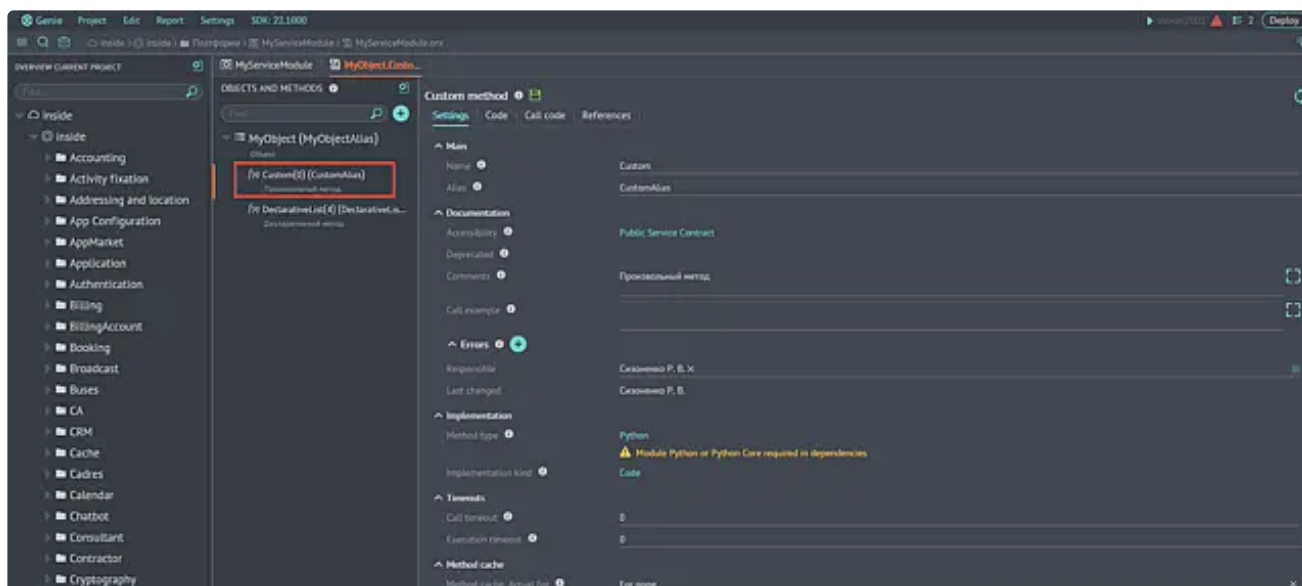
Формат описания произвольного метода объекта

Произвольный метод отмечен тегом `<select>`.

```

1 <select access_mode="0" name="MyObject.Custom"
  alias_name="MyObjectAlias.CustomAlias" is_service="0"
  last_changed="Быканов А.А." last_changed_uuid="3f71a247-c130-4ca5-8497-
  87d107dc08f0" responsible="Быканов А.А." responsible_uuid="3f71a247-
  c130-4ca5-8497-87d107dc08f0" returns="TABLE" type="PYTHON">
2   <comment>Комментарии к методу</comment>
3
4   <!-- Список параметров процедуры -->
5   <!-- Описание тела процедуры -->
6   <definition>
7     <language>PYTHON</language>
8     <body>raise NotImplementedError</body>
9   </definition>
10
11   <!-- Список возвращаемых значений процедуры -->
12   <!-- Список определений процедуры -->
13 </select>

```



Основные атрибуты:

1. **name** – имя (например, «MyObject.Custom»);
2. **alias_name** – псевдоним метода;
3. **access_mode** – для метода «только на чтение»; в параметрах Genie соответствует чекбокс **Readonly** ;
4. **returns** — тип возвращаемого значения – указывает, какой набор данных возвращает метод;
5. **type** — тип метода – определяет способ реализации метода – ХП или функция на C++.
6. **is_service** – является ли метод служебным – служебные методы не видны через RPC.
7. **comment** – пользовательское описание метода.
8. **last_changed** – ФИО разработчика, внесшего изменения в объект;
9. **last_changed_uuid** – uuid разработчика, внесшего изменения.

Все хранимые процедуры создаются в схеме Бизнес-логика, таким образом обращение к серверу БД для вызова процедуры "Документ.Заккрыть" должно происходить так:

"Бизнес-логика" . "Документ .Заккрыть "

Кроме того, процедура может относиться к некоторой категории собственных методов в пределах методов одного объекта (атрибут category), но на структуре БД это никак не отражается.

Формат описания параметров и возвращаемых значений

```

1 <parameter name="integer">
2   <!-- Формат параметра -->
3 </parameter>

1 <return name="string">
2   <!-- Формат возвращаемого значения -->
3 </return>
```

Описание формата параметра и возвращаемого значения полностью совпадает с описанием формата столбца таблицы (тег [<format>](#)) за исключением того, что в описании формата нельзя указывать ограничение *NotNull* и значение по умолчанию.

Формат описания тела процедуры

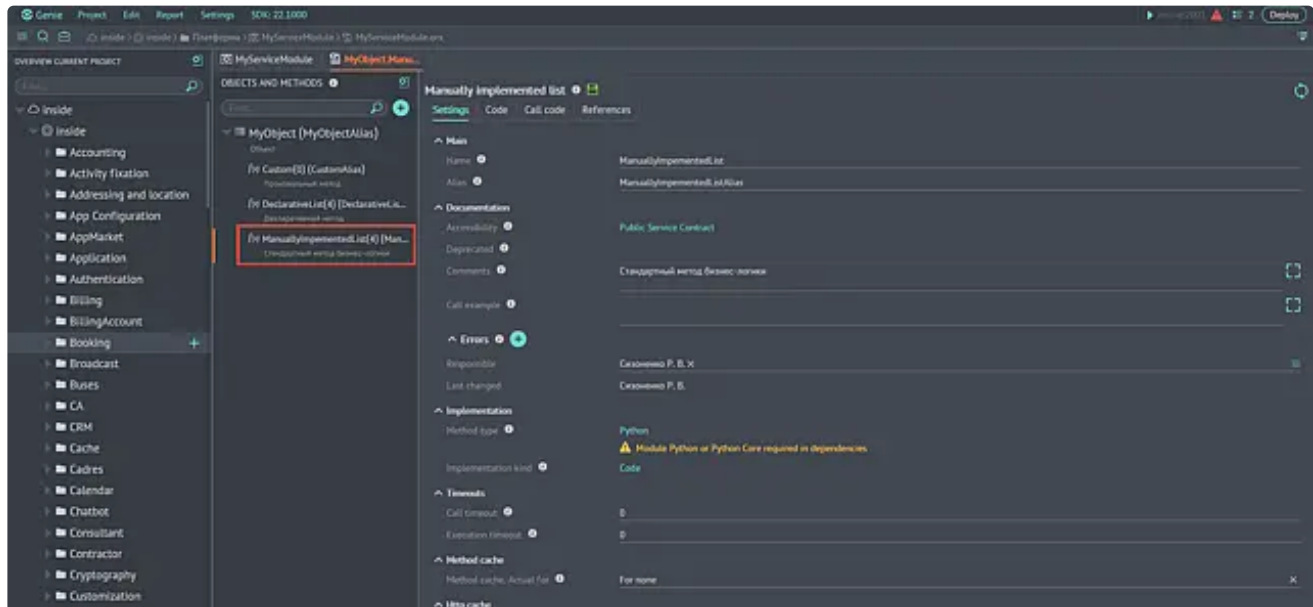
```

1 <definition>
2   <language>PLPGSQL</language>
3   <body>some pl/pgsql code</body>
```

Определение процедуры содержит наименования диалекта SQL (PLPGSQL – для Postgres SQL, TRANSACTSQL – для Ms SQL) и собственно тело процедуры без заголовка (он генерируется автоматически), т.е. начиная с секции DECLARE.

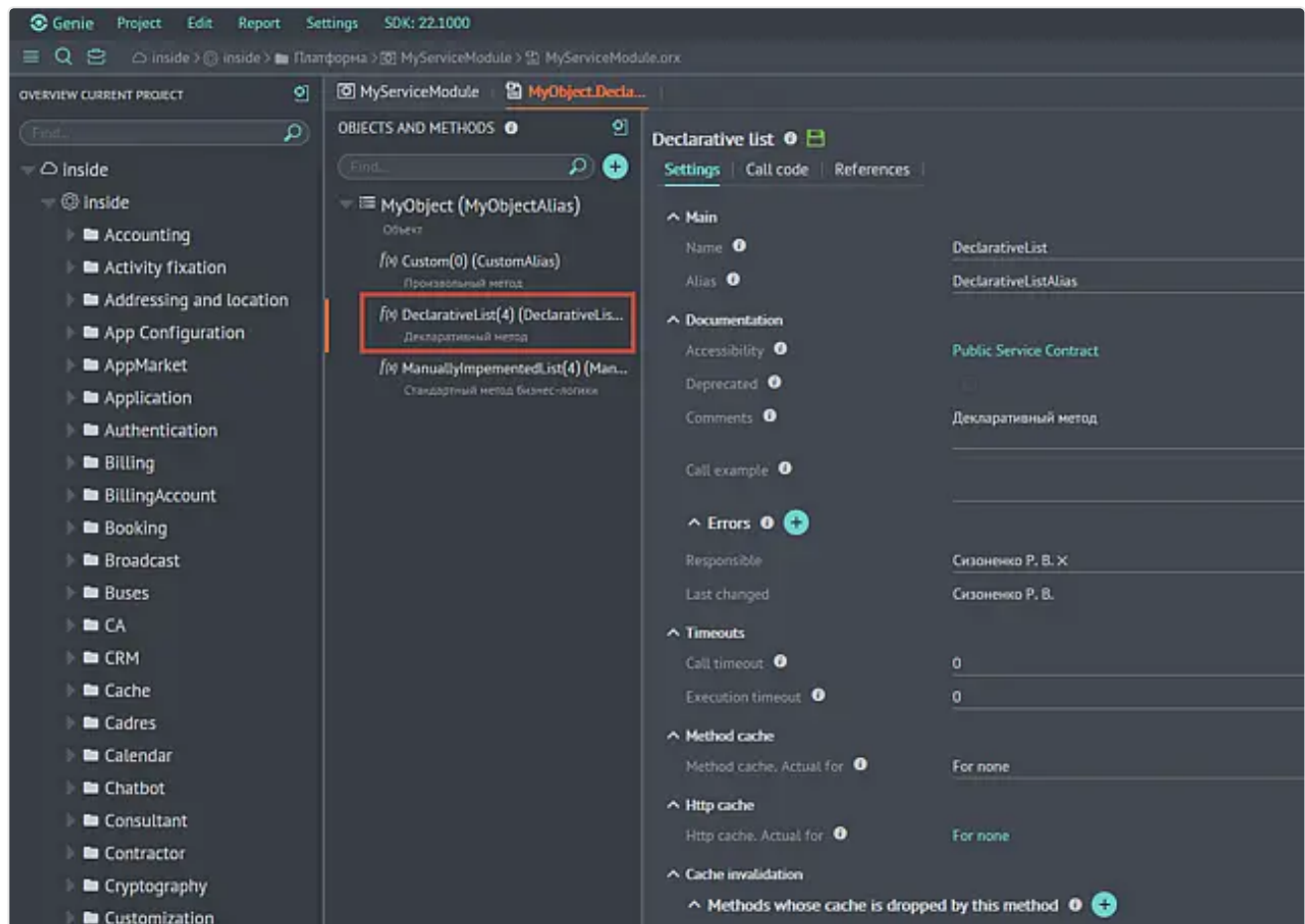
Формат описания стандартного метода бизнес-логики

Формат полностью совпадает с форматом произвольного метода, однако отмечается тегом <standart_select>, а значение атрибута **returns** и список параметров жестко определены.



Формат описания декларативного метода бизнес-логики

Декларативным методам соответствует тег <declarative>.



```

deprecated="1" is_service="0" key="@РедактированиеПоМесту"
last_changed="Быканов А.А." last_changed_uuid="a5a6c71d-0b2e-4e85-9f33-
0f9e04c6d072" name="Отчеты.Список" responsible="Быканов А.А."
responsible_uuid="a5a6c71d-0b2e-4e85-9f33-0f9e04c6d072" type="1">
2   <comment>{"CallExample":"Примеры вызова","Comment":"Это
комментарий","Errors":[{"Error":"Текст ошибки, код","Reason":"Описание
причины ошибки","Solution":"Описание решения ошибки"}]}</comment>
3   <source name="РедактированиеПоМесту"/>
4   <parameter filter_category="SOFTWARE" filter_class="0"
name="ИмяПоляСвязи" param_type="CONSTANT" type="STRING"/>
5 </declarative>

```

1	<declarative accessibility="3" alias_name="List"
2	allfields="true" deprecated="1" is_service="0"
3	key="@РедактированиеПоМесту" last_changed="Быканов
4	А.А." last_changed_uuid="a5a6c71d-0b2e-4e85-9f33-
5	0f9e04c6d072" name="Отчеты.Список"
	responsible="Быканов А.А."
	responsible_uuid="a5a6c71d-0b2e-4e85-9f33-
	0f9e04c6d072" type="1">
	<comment>{"CallExample":"Примеры
	вызова","Comment":"Это комментарий","Errors":
	[{"Error":"Текст ошибки, код","Reason":"Описание
	причины ошибки","Solution":"Описание решения
	ошибки"}]}</comment>
	<source name="РедактированиеПоМесту"/>
	<parameter filter_category="SOFTWARE"
	filter_class="0" name="ИмяПоляСвязи"
	param_type="CONSTANT" type="STRING"/>
	</declarative>

Список атрибутов тега declarative.

Атрибут	Значение	Описание	Примечание
is_service			
	атрибут отсутствует	Опция "Accessibility" со значением "Public".	
	0		
	1	Опция "Accessibility" со значением "Internal".	
module_api_part	1	Опция "Accessibility" со значением "ServiceContract".	Переключает Public, если is_service=0 или не задан.
	0	учитываются настройки атрибута is_service.	
accessibility	3	Опция "Accessibility" со значением "External Stable API".	В настоящий момент другие значения (accessibility=0,1,2) опции не используются, однако в будущем будут соответствовать значениям Internal, Public или Service contract. Теги module_api_part и is_service оставлены для обратной совместимости.
deprecated	1	Метод БЛ помечен как устаревший.	Атрибут используется, если установлен флажок в чекбокс Deprecated.

type	0	Опция "Filter type" устновлена в значение "Free".	
	1	Опция "Filter type" устновлена в значение "Linked records" или "Hierarchical structures".	

Формат описания методов для работы с файлами

Поскольку все методы для работы с файлами на текущий момент на уровне описания почти ничем не отличаются друг от друга, они используют универсальный синтаксис, отличающийся лишь основными тегами.

Пример такого описания:

```

1 <write_file category="Договор"
2         column="Договоры.Договор"
3         name="ТестBLOB.ЗагрузитьДоговор"
4         object="ТестBLOB"
5         type="DEFAULT" />
```

Атрибут **category** – категория метода, аналогично категории в других типах методов, атрибут **column** – имя столбца в структуре записи объекта, с которым работает метод (этот столбец обязательно должен иметь тип «Файл»), атрибут **object** – имя объекта.

Атрибут **type** содержит способ реализации метода:

1. DEFAULT – используется реализация по умолчанию;
2. CUSTOM – используется специализированная реализация.

Основные теги категорий методов:

1. write_file – метод для загрузки файла в базу;
2. write_attach_file – метод для загрузки файла в базу с одновременным прикреплением к объекту;
3. read_file – метод для чтения файла из базы по идентификатору объекта;
4. read_filelist – метод для чтения списка файлов по UUID.